第35卷 第4期 传感技术学报 Vol. 35 No. 4
2022年4月 CHINESE JOURNAL OF SENSORS AND ACTUATORS Apr. 2022

Path Planning Base on Improved D* lite and Time Elastic Band for Mobile Robot*

WU Tao, XIE Zhijun, CHEN Kewei*

(*School of Information Science and Engineering, Ningbo University, Ningbo Zhejiang* 315000, *China*)

Abstract: The path planned by the D* lite algorithm is inefficient, easily affected by moving objects, and close to the known obstacles, which may lead to collision accidents. To deal with these problems, a fusion algorithm combining improved D* lite algorithm and time elastic band method is proposed. Firstly, an improved jump point search strategy is put forward and introduced into the D* lite algorithm, so that the algorithm only accesses the jump point, and this realizes the optimization of both the planning time and the path length. Then the improved time elastic band method is integrated to perform local path planning, which improves the dynamic planning ability of the algorithm and ensures the safety of the planned path. The simulation results show that the improved D* lite algorithm is 12.71%, 65.29%, and 71.80% optimized in terms of path length, planning time and number of path nodes respectively, compared with the original D* lite algorithm. The experimental results show that the fusion algorithm can reduce 11.04% of the path length, 67.10% of the planning time and 11.99% of the movement time compared with the original D* lite algorithm under dynamic conditions, and ensures that the minimum distance between the robot and the obstacle is above a fixed threshold.

Key words: path planning; D* lite; jump point search; time elastic band; mobile robot

EEACC: 7230 **doi: 10.3969/j.issn.1004-1699.2022.04.010**

改进 D* lite 和时间弹性带法的移动机器人路径规划*

吴 涛, 谢志军*, 陈科伟

(宁波大学信息科学与工程学院, 浙江 宁波 315000)

摘 要: 为解决 D* lite 算法在进行路径规划时搜索效率低, 易受运动物体影响及路径贴近障碍物的问题, 提出了一种结合改进 D* lite 算法和时间弹性带法的融合算法。首先, 将改进的跳点搜索引入到 D* lite 算法中, 让算法只对跳点进行访问, 实现了规划时间和路径长度的双优化。之后, 融合改进的时间弹性带法进行局部路径规划, 在提高算法动态规划能力的同时, 保证了规划路径的安全性。仿真结果表明, 改进的 D* lite 算法比原始 D* lite 算法在路径长度、规划时长及路径节点数上分别平均优化了 7.57%、65.29%和 71.80%。在实际表现中, 融合算法在动态情况下比原始 D* lite 算法可减少 11.04%的路径长度、67.10%的规划时长及 11.99%的运动时间, 并且保证了机器人和障碍物间的最小距离。

关键词: 路径规划; D* lite; 跳点搜索; 时间弹性带法; 移动机器人;

中图分类号: TP242 **文献标识码: A** **文章编号: 1004-1699(2022)04-0486-09**

路径规划对于实现移动机器人自主导航具有重要的意义, 其目的是在规定环境中规划出一条由起点到终点的最优路径, 评价指标可以是时间、距离或安全性等[1]。根据对环境的了解程度可分为全局路径规划和局部路径规划[2]。全局路径规划常见的有迪杰斯特拉算法[3]、A* 算法[4]、跳点搜索算法[5]、D* lite 算法[6]、RRT* 算法[7]等。迪杰斯特拉算法、A* 算法及跳点搜索算法多用于静态环境下。迪杰斯特拉算法利用贪心策略得到起始点到地图上全部点的最短距离, 因此在大型地图上会消耗较多算力。A* 算法通过比较当前节点的八邻域启发函数值搜索路径, 提高了搜索的效率, 但是当八邻域中出现相同的最小值时, 可能无法求出最优解。而跳点搜索算法是在 A* 算法的基础上只对跳点进

项目来源: 国家自然科学基金重点资助项目(U20a20121); 浙江省自然科学基金资助项目(LY21F020006); 衢州市科技计划基金资助项目(2020K06)

收稿日期: 2021-09-11 修改日期: 2021-11-22

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

行搜索[8],进一步提高了搜索效率。然而上述 3 种算法在动态环境下都难以保证效果。D* lite 算法虽适用于动态环境,但算法效率却有待提高。RRT* 算法也可用于动态的路径规划,其优势在于对环境建模要求不高,可在不同的地图模型上进行规划;但是其规划精度和迭代次数有较大关系,更高的精度意味着更加耗时。

局部路径规划算法常见的有动态窗口法[8]、粒子群算法[9]、时间弹性带法[10]等。动态窗口法可实现实时避障,通过在一个动态的窗口中估计机器人可能的轨迹决定机器人的下一个动作,但在复杂的环境下对于轨迹的评估将变得十分复杂,使得效果下降。粒子群算法将每一条规划的路径视为一个粒子,通过对粒子群进行演化,选择最优的粒子个体,该算法随机性较大,同样不适合复杂环境。时间弹性带法通过配置多个优化目标,包括路径长度、规划时长、安全距离以及运动学约束等,利用图优化的方式进行问题的求解,但该算法对于全向机器人的支持较差[11]。

D* lite 算法(以下简称 D* lite)是一种增量式启发算法,相比于广泛使用的 A* 算法,更适用于在未知环境下的路径规划。Setiawan 等[12]通过对 A* 算法和 D* lite 进行仿真和试验的对比,得出结论:D* lite 在动态环境下,多数情况要好于 A* 算法。由于 D* lite 还存在一些问题,各种改进方法也被提出,Xie 等[13]提出了一种改进的 D* lite,通过子节点选择优先级策略,由垂直方向的节点状态决定对角方向的节点是否展开,使得规划的路径与障碍物之间产生了安全距离,但该方法会增加节点数量导致非最优解的出现。Yao 等人[14]提出了一种将 D* lite 和人工势场法相结合的融合算法,并将其应用于复杂环境下的无人船自动驾驶,仿真实验表明该算法规划的路径平滑且安全,但是该算法并未对 D* lite 的规划效率进行优化。Li 等人[15]将D* lite 和其提出的人工不可遍历顶点的概念相结合,提出了一种可以快速重规划的多机器人路径规划算法,在检测到碰撞发生前可基于人工不可遍历顶点和 D* lite 进行路径的重规划。黄鲁等[16]提出了一种基于路径优化的 D* lite,通过将懒惰视线法和距离变换引入 D* lite 中,使得算法可以在任意方向进行拓展,同时和障碍物保持了一定的安全距离,但是这样会降低算法的搜索效率。刘军等[17]通过观察目标点与障碍物之间的关系,引入关键节点概念来降低搜索空间,让算法只对障碍物附近的关键点进行访问,路径由相互连接的关键点构成,很好地降低了搜索空间,但是产生了路径贴近障碍物的问题。

由于机器人在实际应用中需要考虑的因素更为复杂,单一的全局路径规划常常难以胜任。全局路径规划和局部路径规划相结合成为一种更为常见的做法。当前和全局规划算法结合较多的局部规划算法有动态窗口法及时间弹性带法。庞磊等[18]将 A* 算法和时间弹性带法相融合,通过激光点云标记非行人动态信息,生成全局静态地图和局部动态地图。利用 A* 算法在静态地图中进行全局规划,时间弹性带法在局部地图中进行动态规划实现了行人目标的动态跟踪。李秀智等[19]提出了基于复合式协同策略的机器人自主探索方法,利用改进的 RTT 算法进行机器人自主探索,将 A* 算法和时间弹性带法相融合进行路径规划。通过增加约束条件,解决机器人倒退时容易碰撞到障碍物的问题,实现了机器人自主探索及路径规划。

针对以上提到的问题,本文提出一种结合改进 D* lite 和时间弹性带法的融合算法,通过全局优化提高搜索效率,减少冗余节点,通过局部优化降低动态物体对算法规划效率的影响,并提高路径安全性。

1　全局路径优化

1.1　D* lite

D* lite 采用一种从目标点向起始点规划的策略,并适用于起始点改变的情况。算法流程如图 1 所示。

D* lite 在初始化地图后,首先预规划出一条最优路径。当机器人运动后,每运动一次就判断当前位置是否为目标点。若为否,从优先列表中选出

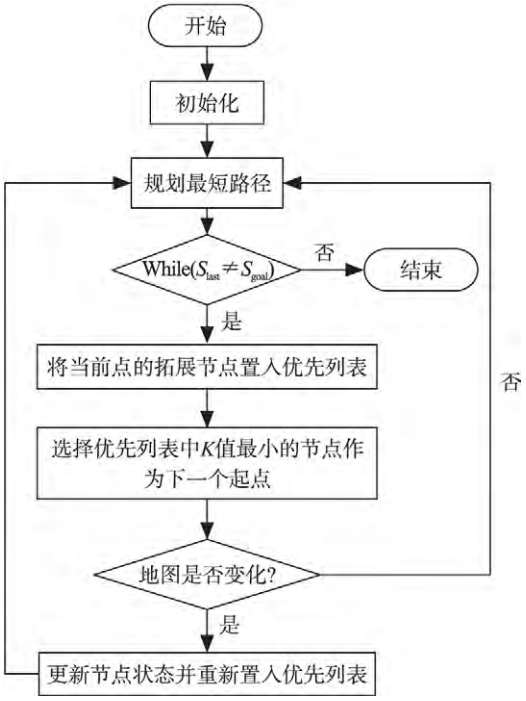


图 1　D* lite 流程图

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

k 值最小的节点作为下一个起点。当路径出现障碍物后，将路径上受影响的点更新状态后再次置入优先列表中，并重新从目标点开始规划出一条路径。由于采用了先前的节点信息，可以实现较快的重规划。

在生成最优路径这个阶段，D* lite 会在地图上建立一个“路径场”，增量式的靠近目标。使得算法资源消耗巨大，同时会产生一些冗余拐点。

为了提高规划效率，在 D* lite 拓展路径节点前增加一个筛选跳点的过程，减少对一些无关节点的计算和存储。

1.2 跳点筛选策略

传统 D* lite 拓展节点时，是在当前节点的八邻域中找到下一个节点，同时将这些点都存储在优先列表中，导致搜索效率下降。而传统跳点搜索策略也会在对角方向产生冗余，本文通过在对角方向搜索时，增加一个判定过程以消除这种冗余。具体可分为 2 种情况：

情况 1　遍历的方向为水平或竖直

①在这两个方向上的节点若为起始节点或者目标点，则认为是关键点。

②若在这两个方向上的某一节点有强迫邻点则认为是关键点。

如图 2 所示，节点 Y 的八邻域中存在障碍物，且节点 X 经过 Y 到达节点 N 的距离代价相比于不经过 Y 到达 N 的任意路径的距离代价小，就认为 Y 有强迫邻点 N。

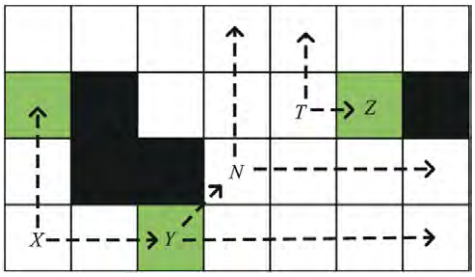


图 2　关键点筛选策略

情况 2　当遍历方向为对角方向

当沿着对角方向搜索时，若对角方向的节点在水平或者垂直方向上的延伸节点满足情况 1，且这些满足情况 1 的延伸节点都与出发点间无障碍物，则将这些延伸节点置入优先列表。如图 2 所示，从出发点 Y 开始进行对角方向的搜索，在 T 节点的水平方向上有节点 Z 是个关键点，且节点 Z 与出发点 Y 之间无障碍物，则将 Z 置入优先列表。值得注意的是，情况 2 时，对角节点在水平或竖直方向的符合条件 1 的全部点都需与遍历的出发点间没有障碍物，才可舍弃对角方向上的点，否则必须保留对角方向上的点。

1.3 优化搜索步长

在每次搜索关键点时都会从当前节点出发一直遍历到关键点或者边界。当地图较大时，会导致搜索效率的下降。可通过计算当前点和起始点的距离，动态地调整搜索的步长，提高搜索效率。常见的距离算法有欧式距离、对角距离和曼哈顿距离。为确保最大可能地搜索到关键点，本文选用曼哈顿距离计算搜索步长，公式如下：

$$d(c,s)=|x_c-x_s|+|y_c-y_s| \tag{1}$$

式中：$d(c,s)$ 表示搜索的步长，c 和 s 分别表示当前点和起始点。当在搜索步长内未发现关键点时，则将步长依次乘以一个倍数 $n\in\left\{2,3,\cdots,\frac{L}{d(c,s)}\right\}$，继续进行搜索，$L$ 为地图的最大边长。

2 局部路径优化

针对动态物体对 D* lite 规划效率影响大、规划路径安全性低的问题。本文通过改进的时间弹性带法进行局部规划加以解决，当时间弹性带法无法完成局部规划时，再调用改进 D* lite 进行重规划。

2.1 时间弹性带法及其约束

时间弹性带法是一种适用于竞速机器人的局部路径规划算法，具有规划速度快、参数配置灵活的特点。但是对于可以原地旋转的机器人会产生不必要的倒退及转折，增加运动时间，在某些情况下甚至会导致路径规划失败[20]。本文通过增加转向约束来避免以上情况的发生。

定义机器人在时刻 k 的位姿为：

$$S_k=[x_k,y_k,\beta_k]\in\mathbb{R}^2\times S^1 \tag{2}$$

式中：x_k、y_k 为机器人在地图坐标系中的坐标，$\beta_k\in S^1$，$S^1\in[-\pi,\pi]$ 为机器人的偏航角。机器人的位姿序列可表示为：$Q=\{S_k\}_{k=1,\cdots,n}$。设相邻的两个位姿间的时间间隔为 ΔT，则时间序列可表示为 $\Delta T=\{\Delta T_k\}_{k=1,\cdots,n-1}$，其中 ΔT_k 表示从位姿 S_k 到位姿 S_{k+1} 所用的时间。将位姿和时间序列合并可得到时间弹性带 $b=\{Q,T\}$。图 3 为三个连续的时间弹性带。

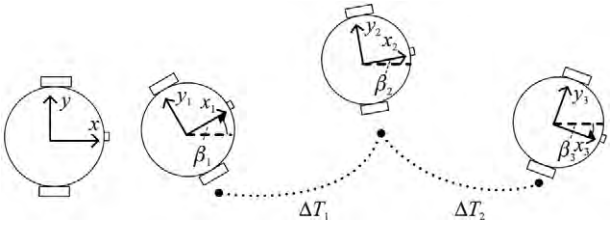


图 3　时间弹性带序列

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

而时间弹性带法的优化模型可以表述为找到在最短时间内让机器人从初始位姿 S_1 转换到目标位姿 S_n 的一系列路径点,同时需要满足运动学上的约束及保证机器人的安全。该过程可以转化为求解一个非线性的函数:

$$V^*(b)=\min_{Q}\sum_{k=1}^{n-1}\Delta T_k^2 \tag{3}$$

它由以下约束限制:

$$S_1=S_{\text{start}},S_n=S_{\text{final}},\Delta T>0$$
$$v_k(S_{k+1},S_k,\Delta T_k)\geqslant 0,(k=0,1,\cdots,n)$$
$$O_k(S_k)\geqslant 0$$
$$\alpha_k(S_{k+2},S_{k+1},S_k,\Delta T_{k+1},\Delta T_k)\geqslant 0,(k=1,2,\cdots,n)$$
$$\alpha_1(S_1,\Delta T_1)\geqslant 0,\alpha_n(S_n,S_{n-1},\Delta T_{n-1})\geqslant 0$$

式中: 初始位姿 S_1 和最终位姿 S_n 分别受到 S_{start} 和 S_{final} 的约束。机器人的速度及加速度分别由不等式 $v_k(\cdot)$ 及 $\alpha_k(\cdot)$ 约束。不等式 $O_k(S_k)$ 约束了机器人和障碍物之间的最小距离。

2.2　速度和加速度约束

机器人的平均线速度 $\boldsymbol{v}_k$ 和平均角速度 ω_k 可以由两个连续位姿 S_{k+1},S_k 的距离范数、偏航角的差值及对应的时间间隔 ΔT_k 分别计算得到,如式(4)和式(5)所示:

$$\boldsymbol{v}_k=\Delta T_k^{-1}\left\|\left[x_{k+1}-x_k,y_{k+1}-y_k\right]^{\mathrm{T}}\right\| \tag{4}$$

$$\boldsymbol{\omega}_k=\Delta T_k^{-1}(\beta_{k+1}-\beta_k) \tag{5}$$

令 $\boldsymbol{v}_{\max},\boldsymbol{\omega}_{\max}$ 分别表示机器人的最大线速度和最大角速度。则速度的约束可用式(6)表示:

$$\boldsymbol{v}_k(s_{k+1},s_k,\Delta T_k)=\left[\boldsymbol{v}_{\max}-|\boldsymbol{v}_k|,\omega_{\max}-|\omega_k|\right]^{\mathrm{T}} \tag{6}$$

机器人的平均线加速度 $\boldsymbol{a}_k$ 和平均角加速度 $\dot{\omega}_k$ 表示如下:

$$\boldsymbol{a}_k=\frac{2(\boldsymbol{v}_{k+1}-\boldsymbol{v}_k)}{\Delta T_k+\Delta T_{k+1}} \tag{7}$$

$$\dot{\omega}_k=\frac{2(\omega_{k+1}-\omega_k)}{\Delta T_k+\Delta T_{k+1}} \tag{8}$$

令 $\boldsymbol{a}_{\max},\dot{\omega}_{\max}$ 分别表示机器人的最大线加速度和最大角加速度。则加速度的约束可用式(9)表示:

$$\boldsymbol{a}_k(s_{k+2},s_{k+1},s_k,\Delta T_{k+1},\Delta T_k)=\left[\boldsymbol{a}_{\max}-|\boldsymbol{a}_k|,\dot{\omega}_{\max}-|\dot{\boldsymbol{\omega}}_k|\right]^{\mathrm{T}} \tag{9}$$

2.3　躲避障碍物

设机器人在位姿 S_k 时,局部区域的障碍物集合为 O,o_i 表示集合中的障碍物,$i=1,2,\cdots,R$。与障碍物的最小间距设为 $\rho_{\min}$。则机器人与障碍物之间的约束可表示为:

$$O_k(S_k)=\left[\rho(S_k,o_1),\rho(S_k,o_2),\cdots,\rho(S_k,o_R)\right]^{\mathrm{T}}-\left[\rho_{\min},\rho_{\min},\cdots,\rho_{\min}\right]^{\mathrm{T}} \tag{10}$$

2.4　转向约束

本文研究的是可原地旋转的全向机器人,利用式 11 来提取机器人运动的方向。

$$\gamma(S_k,S_{k+1})\approx\frac{\kappa\langle\boldsymbol{q}_k,\boldsymbol{d}_{k,k+1}\rangle}{1+|\kappa\langle\boldsymbol{q}_k,\boldsymbol{d}_{k,k+1}\rangle|} \tag{11}$$

式中: $\boldsymbol{q}_k=[\cos\beta_k,\sin\beta_{k+1},0]^{\mathrm{T}}$,而 κ 是一个可变的比例因子(本文中取 $\kappa=10^2$),$\boldsymbol{d}_{k,k+1}$ 表示两个位姿间的距离向量,符号<>计算向量的混合积。$\gamma(S_k,S_{k+1})$ 的取值为$\{-1,1\}$。当取值为 -1 时机器人向后运动。当机器人需要向后运动时,通过建立转向约束,使得机器人进行原地旋转。约束如下:

$$\tau_k(S_k,S_{k+1})=\left[\frac{\beta-\beta_0}{\omega_{\max}}\right] \tag{12}$$

式中: β 表示全局规划路径上离机器人最近的点和机器人 x 轴的夹角,β_0 为机器人当前位姿的偏航角。$\omega_{\max}$ 为机器人最大的角速度。

3　算法融合

改进 D* lite 和时间弹性带法融合后,在拥有动态规划能力的基础上,提高了搜索效率及路径的安全性。融合步骤为: 改进 D* lite 规划全局路径,得到全局路径点。将两个相邻的路径点确定为局部路径规划的起始点和目标点。之后,进入到局部规划中,若局部规划成功,判断是否为全局的终点; 否则调用改进 D* lite 进行重规划。融合算法流程图如图 4 所示。

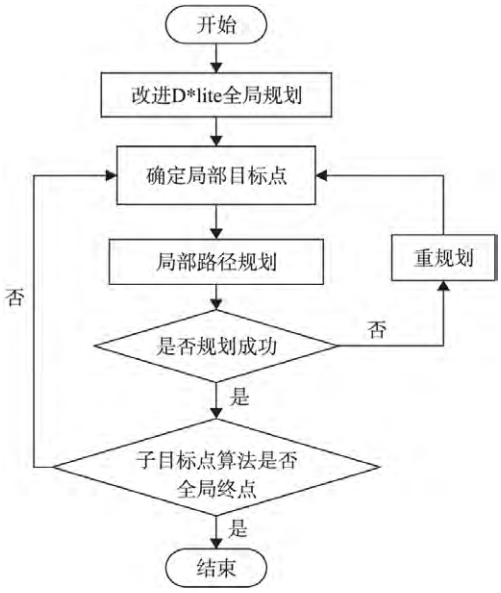


图 4　融合算法流程图

4　仿真及实验验证

为了验证改进算法的有效性,对本文提出的改

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

进 D* lite(以下简称 JD* lite)、改进时间弹性带法及融合算法分别进行仿真实验。之后,将融合算法应用在真实的物理环境中进行实验验证。硬件平台为:Intel Core i5 8400 处理器,16 GB 内存;编程语言 Python,C++。

4.1 全局仿真实验

在 20×20 及 200×200 的栅格地图上分别进行随机的障碍物填充,各生成 10 份不同的地图。规定起点为(1,1),终点分别为(20,20)及(200,200)。分别用 D* lite、RRT* 及 JD* lite 进行仿真实验。并设定 RRT* 在两种情况下的迭代次数分别为 2 000次及 15 000 次。仿真实验结果如图 5、图 6 所示。

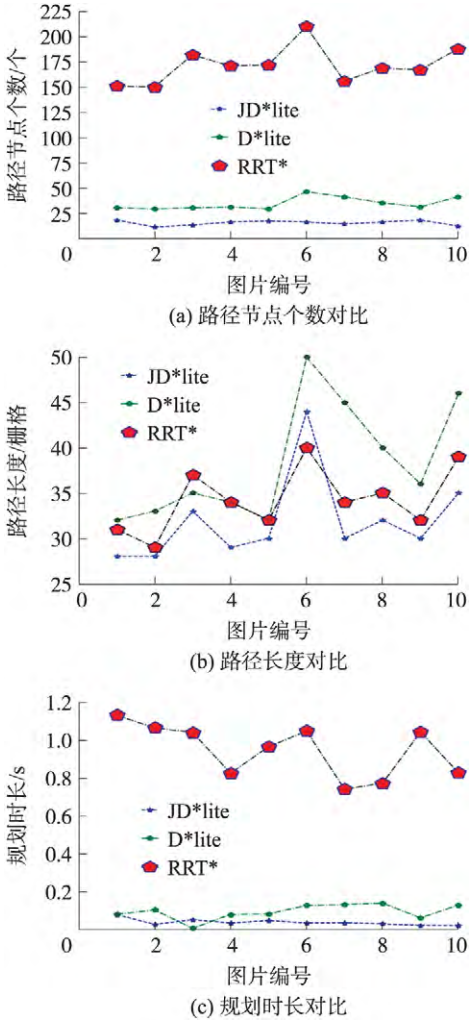


图 5 20×20 地图下 3 种算法数据对比

由图 5(a)及图 6(a)可以看出,RRT* 的路径节点个数在两种情况下都要多于另外两种算法。这是因为 RRT* 不是以单个栅格进行子节点的拓展,单个栅格中会出现多个拓展节点,这大大增加了节点的个数。而 JD* lite 在路径节点个数上体现出较大的优势,两种情况下的平均节点个数仅为 15 个及

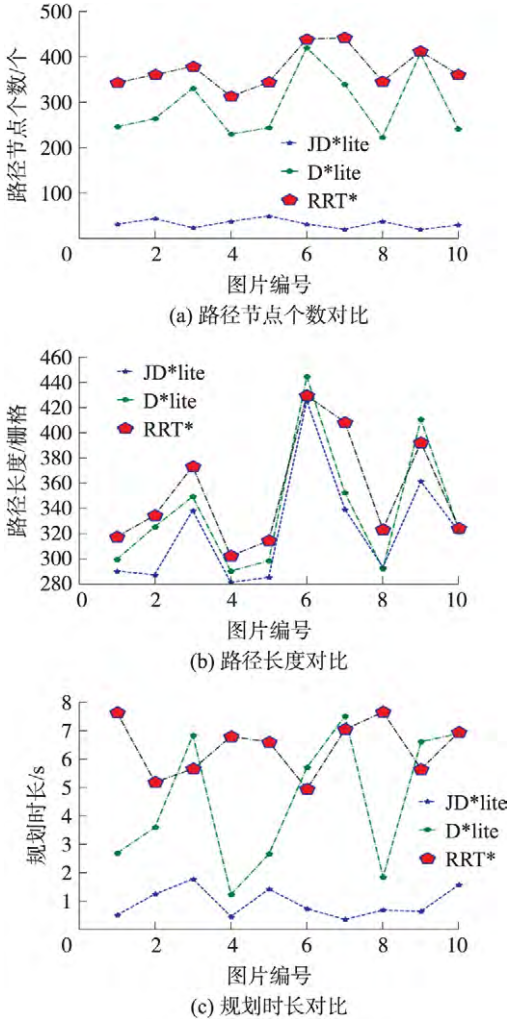


图 6 200×200 地图下 3 种算法数据对比

32 个。远低于 RRT* 的 162 个、382 个和 D* lite 的 34 个、293 个。这是因为 JD* lite 在 D* lite 的基础上,引入了跳点搜索策略来搜寻子节点,同时删除路径上的冗余跳点,使得 JD* lite 在栅格地图上的路径节点数为三者最优。

在规划的路径长度上,观察图 5(b)、图 6(b)可知,JD* lite 在大多数情况下处于领先。在两种情况下的仿真实验中,仅有一次路径长度长于 RRT* ,其他均优于 D* lite 及 RRT* 。平均优于D* lite规划长度 7.57%及 RRT* 规划长度 9.83%。

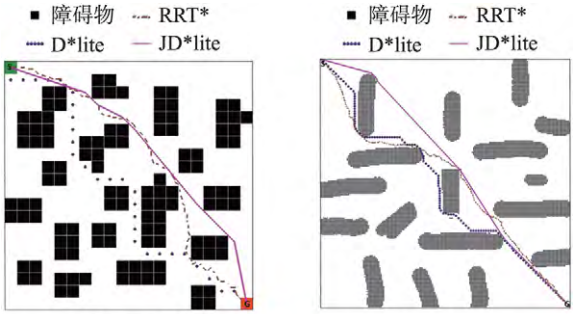
出现上述现象的原因是 D* lite 采用八邻域做路径规划,而 JD* lite 可跳出八邻域的限制,实现更加灵活的规划。同时删除了冗余的拐点使得路径进一步缩短。而 RRT* 的规划方式虽然更加灵活,但是需要迭代的次数也大大增加,在本文设定的迭代次数下,效果弱于 JD* lite。

在规划时间方面,20×20 地图下 D* lite 和 JD* lite的规划时长差距不大,平均时间为 0.04 s 和 0.10 s。而在 200×200 的地图上时,则产生了较大的

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

差距,平均时间分别为 0.93 s 和 4.55 s。这是因为 JD* lite 遍历地图时仅需要判断该点是否为跳点,无需存储大量非关键点,每次进行子节点拓展时需要根据节点的关键值进行排序操作,而这个过程较为耗时。减少排序队列的长度可有效降低规划时长。RRT* 的规划时长为 3 者最长,原因是 RRT* 在迭代次数固定的情况下,其规划速度和规划地图的难度有较大的关系,当遇到狭窄的通道时,会明显降低规划速度。

两种情况下各随机选出一份规划路径图进行比较,见图 7。可以看到三种算法都能规划成功,D* lite规划的路径有较多的转折,从而导致路径冗余。而 RRT* 生成的路径较为曲折,不适合直接用于机器人的路径导航。而观察图 7 发现 D* lite 规划的路径总是贴近障碍物的拐角处,这容易导致机器人的碰撞事故发送,需要结合局部路径规划,进行安全性优化。



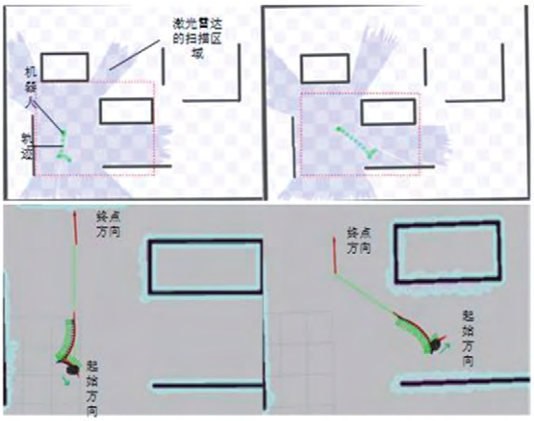
(a) 20×20环境　　(b) 200×200环境

图 7　3 种算法的规划结果

4.2　改进时间弹性带法有效性仿真实验

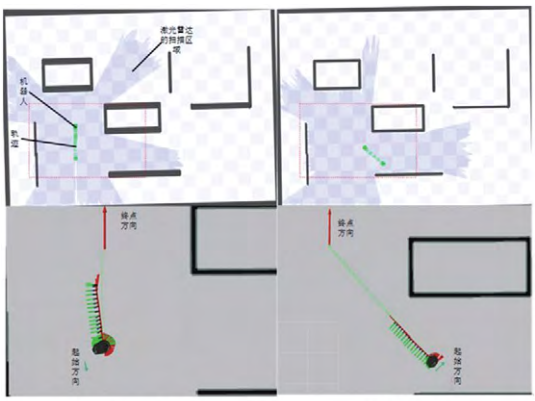
为了验证时间弹性带法改进的有效性,本文使用 Stage 仿真平台进行仿真实验。首先,建立如图 8 所示的仿真环境,并导入配备了激光雷达的 Turtlebot2 机器人模型。

如图 8 和图 9 所示,图中第 1 行为虚拟仿真环境。下面第 2 行为第 1 行中虚线区域的地图及规划路径放大图。从图 8 中可以看到,当目标点在机器人正方向的侧面及后面时,机器人会产生倒退的路径。在图 9 中,改进后的时间弹性带法在原地调整自身的位姿后,克服了上述的问题。



(a) 局部路径一　　(b) 局部路径二

图 8　未优化前的时间弹性带法



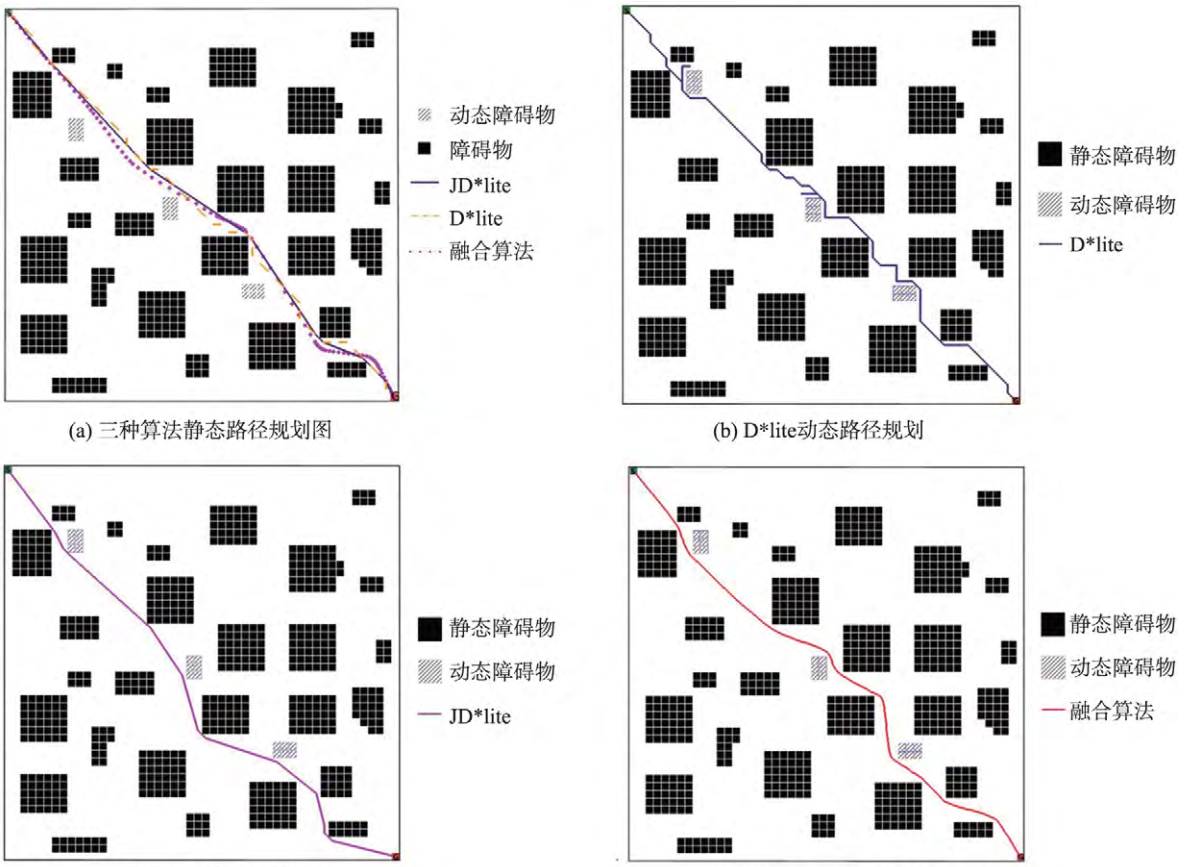
(a) 局部路径一　　(b) 局部路径二

图 9　优化后的时间弹性带法

4.3　融合算法有效性仿真实验

为验证本文提出的融合算法的有效性,分别对 D* lite,JD* lite 及融合算法在环境相同的栅格地图上进行路径规划,并设置动态障碍物。动态障碍物在水平或者竖直方向做往复的匀速运动。图 10(a) 为三种算法未与动态障碍物相遇时的情况,结合表 2 可知,JD* lite 在规划路径长度和规划速度上处在领先的地位,同时发现 JD* lite 规划的路径贴近障碍物,存在安全隐患。而融合算法由于加入了局部规划,在规划时间上比 JD* lite 要多; 同时考虑了和障碍物的最小距离,使得路径长度也有所增加,但是依然比原始的 D* lite 提升了 4.21%、45.58%。图 10(b) 为动态障碍物运动到 D* lite 预规划好的位置上后重规划生成的路径。可以看到 D* lite 成功地进行了重规划,但是由于动态障碍物的运动导致规划路径变的不稳定,产生了路径的分叉。观察图 10(c) 并结合表 1 可知,JD* lite 在动态物体的扰动下依然可以规划出路径,同时可以发现规划路径的距离变大了。这是因为动态物体使搜索的跳点位置发生变动,选择的跳点可能不是真正的最优点。结合图 10(d) 及表 2 可知,当在已规划好的路径上出现动态障碍物时,融合算法可在局部路径上进行局部动态规划,避开动态障碍物之后,回到原来的路径上,同时规划的路径更加平滑,符合机器人的运动约束。在保持最优路径的同时,做到安全高效的规划。由表 2 可以看到局部路径规划的重规划用时要比 JD* lite 的用时要长,但是规划的路径长度更接近原始长度。

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net



(a) 三种算法静态路径规划图 (b) D*lite动态路径规划

(c) 改进D*lite动态路径规划 (d) 融合算法动态路径规划

图 10 动态路径规划图

表 1 静态路径规划性能对比

算法	路径长度	预规划用时/ms
D* lite	82.22	1121
JD* lite	77.76	311
融合算法	78.83	610

表 2 动态路径规划性能对比

算法	路径长度	重规划用时/ms
D* lite	87.43	532
JD* lite	85.35	123
融合算法	80.87	315

4.4 融合实验与分析

为进一步验证本文提出的融合算法在移动机器人路径规划上的可行性，利用机器人操作系统(Robot Operating System，ROS)在真实环境下进行实验。系统环境为 Ubuntu 16.04 操作系统，实验机器人为 Turblebot2 机器人，如图 11(a)所示。

本文的实验场地为办公室走廊到电梯口的一部分，如图 11(b)所示。通过机器人所配置的激光雷达建立的地图如图 11(c)所示。

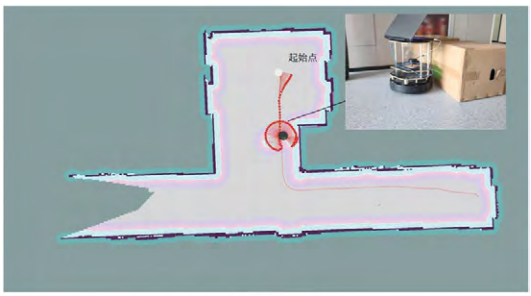
(a) 实验平台 (b) 实验环境 (c) 实验地图

图 11 实验环境及建图

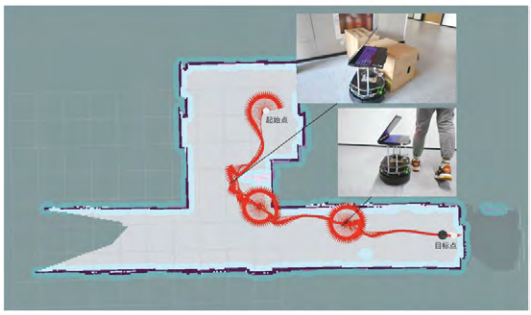
分别运行 A* 算法、D* lite、JD* lite 及融合算法，得到的路径如图 12 所示，图 12(a)为 A* 算法规划的路径，可以发现在规划好的路径上遇到未知障碍物时，A* 算法不能绕过未知障碍物，导致路径规划失败。图 12(b)为传统 D* lite 算法生成的路径，传统的 D* lite 绕过未知障碍物和动态障碍物后到达了目标点，但是从运动过程中可以看到，机器人会与障碍物发生碰撞，造成危险，这是由于 D* lite 算法本身并未考虑机器人和障碍物的距离。图 12(c)为 JD* lite 规划生成的路径图，其规划的路径比起 D* lite 算法减少了一些多余的拐点，路径的转折角度幅度更大。同时发现规划路径和障碍物的距离较近，虽然最终也绕过了障碍物，但是在这个过程中出现了一些停顿，并且在运动过程中还和障碍物发生了轻微的碰撞。图 12(d)为融合算法生成的路径，可以看到融合

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

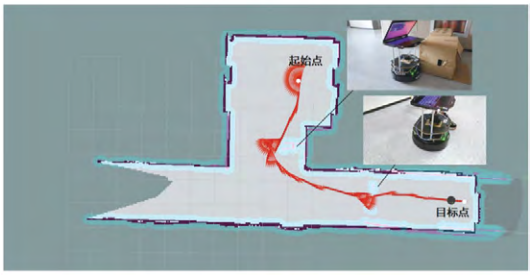
算法生成的路径较为平滑,且与障碍物始终可以保持一个最小的距离,在遇到动态障碍物时成功地进行了局部动态规划,避开了正在运动的物体。



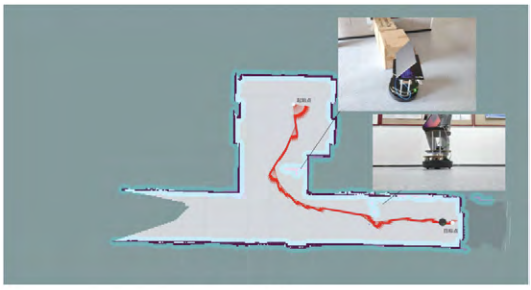
(a) A*算法



(b) D*lite算法



(c) JD*lite



(d) 融合算法

图 12　对比实验

表 3 给出 D* lite、JD* lite 及融合算法在图 12 地图中的性能表现。

表 3　三种算法性能

指标	D* lite	JD* lite	融合算法
规划时长/ms	3 772	996	1241
路径长度/m	15.4	13.4	13.7
最小距离/m	0	0	0.28
运动时长/s	107.54	102.9	94.65

由表 3 可知,融合算法相比 JD* lite 在时间及长度上虽然有所增加,但是提高了机器人的安全性。相比于 D* lite 在规划时长、路径长度、最小距离及运动时长上分别优化了 67.10%、11.04%、100%和 11.99%。

5　结论

本文通过将改进的跳点搜索引入到 D* lite 中进行关键点的筛选,融合时间弹性带法进行局部路径优化,提高了规划效率、提升了动态情况下的规划能力,同时提高了生成路径的安全性。算法在真实的环境下获得了较为明显的优化效果。

需要指出的是本文算法研究的是以栅格地图为基础的路径规划,规划结果的精度是由栅格的大小决定的。当选用较小栅格尺寸的模拟地图环境时会导致算法的性能有所下降。同时,本文的实验环境较为简单,在复杂环境中大量的动态物体对于算法的稳定性要求也会更加严格。同时由于设置了和障碍物的安全距离约束,致使机器人在狭窄通道中的运动受到一定限制。

在接下来的研究中,将尝试复杂动态场景下的移动机器人路径规划研究,进一步推动移动机器人路径规划的研究与实践。

参考文献:

[1] 霍凤财,迟金,黄梓健,等. 移动机器人路径规划算法综述[J]. 吉林大学学报: 信息科学版,2018,36(6):639-647.

[2] Faridi A Q,Sharma S,Shukla A,et al. Multi-Robot Multi-Target Dynamic Path Planning Using artificial Bee Colony and Evolutionary Programming in Unknown Environment [J]. Intelligent Service Robotics,2018,11(2):171-186.

[3] Dijkstra E W. A Note on Two Problems in Connexion With Graphs [J]. Numerische Mathematik,1959,1(1):269-271.

[4] Hart P E,Nilsson N J,Raphael B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths [J]. IEEE Transactions on Systems Science and Cybernetics,1968,4(2):100-107.

[5] Harabor D,Grastien A. The JPS Pathfinding System [C]//5th Annual Symposium on Combinatorial Search. Menlo Park, USA: AAAI,2012: 207-208

[6] Souissi O,Benatitallah R,Duvivier D,et al. Path Planning: A 2013 Survey [C]//International Conference on Industrial Engineering and Systems Management. Agdal,Morocco,IEEE,2013: 1-8.

[7] Chao N,Liu Y K,Xia H,et al. Grid-based RRT* ,for Minimum Dose Walking Path-Planning in Complex Radioactive Environments [J]. Annals of Nudea Energy,2018,115(1):73-82.

[8] 赵晓,王铮,黄程侃,等. 基于改进 A* 算法的移动机器人路径规划[J]. 机器人,2018,40(6):903-910.

[9] Fox D,Burgard W. The Dynamic Window Approach to Collision A-

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

voidance [J]. IEEE Robotics & Automation Magazine, 1997, 4(1): 23-33.

[10] Rendón M A, Martins F F. Path Following Control Tuning for an Autonomous Unmanned Quadrotor Using Particle Swarm Optimization [J]. IFAC Proceedings Volumes, 2017, 50(1): 325-330.

[11] 文郁,黄江帅,江涛,等. 安全平滑的改进时间弹性带轨迹规划算法[J/OL]. 控制与决策: 1-9 [2021-11-14]. https://doi.org/10.13195/j.kzyjc.2021.0068.

[12] Setiawan Y D, Pratama P S, Jeong S K, et al. Experimental Comparison of A* and D* Lite Path Planning Algorithms for Differential Drive Automated Guided Vehicle [J]. In: Zelinka I, Duy V, Cha J. (eds) AETA 2013: Recent Advances in Electrical Engineering and Related Sciences. Lecture Notes in Electrical Engineering. Springer, Berlin, Heidelberg, 2014, 282(1): 555-564.

[13] Xie K, Qiang J, Yang H. Research and Optimization of D-Start Lite Algorithm in Track Planning [J]. IEEE Access, 2020, 8(1): 161920-161928.

[14] Yao Y L, Liang X F, Li M Z, et al. Path Planning Method Based on D* lite Algorithm for Unmanned Surface Vehicles in Complex Environments [J]. China Ocean Engineering, 2021, 35(3): 372-383.

[15] Li H, Zhao T, Dian S. Prioritized Planning Algorithm for Multi-Robot Collision Avoidance Based on Artificial Untraversable Vertex [J]. Applied Intelligence, 2021, 52(1): 1-23.

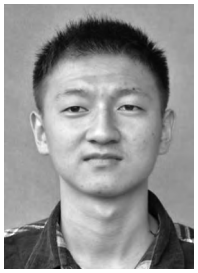
[16] 黄鲁,周非同. 基于路径优化 D* Lite 算法的移动机器人路径规划 [J]. 控制与决策, 2020, 35(4): 112-119.

[17] 刘军,冯硕,任建华. 移动机器人路径动态规划有向 D* 算法 [J]. 浙江大学学报(工学版), 2020, 54(2): 291-300.

[18] 庞磊,曹志强,喻俊志. 基于 A* 和 TEB 融合的行人感知无碰跟随方法 [J]. 航空学报, 2021, 42(4): 501-510.

[19] 李秀智,赫亚磊,孙炎珺,等. 基于复合式协同策略的移动机器人自主探索 [J]. 机器人, 2021, 43(1): 44-53.

[20] Rösmann C, Feiten W, Wösch T, et al. Efficient trajectory optimization using a sparse model [C] //2013 European Conference on Mobile Robots, Barcelona, Spain, IEEE, 2013: 138-143.



吴 涛(1994—),男,宁波大学信息科学与工程学院计算机技术在读硕士研究生,主要从事机器人路径规划的研究,wutao184@163.com;

谢志军(1974—),男,通讯作者,工学博士,宁波大学信息科学与工程学院副教授,硕士生导师,主要从事无线传感器网络、物联网的研究,xiezhijun@nbu.edu.cn;

陈科伟(1963—),江苏无锡人,现任宁波大学教授、硕博导师,主要研究方向为智能制造、机器人、计算智能,chen_kewei@nbu.edu.cn。

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net